

Introduction to Image Segmentation

Segmentation is the process of **partitioning** a digital image into multiple regions and extracting the meaningful region which is known as **Region of Interest (ROI)**

- ❑ Region of Interest (ROI) vary with applications
- ❑ In fact no single universal segmentation algorithm exists for segmenting the ROI in all images
- ❑ Therefore many segmentation algorithms need to apply and pick that algorithm which performs the best for given requirement

Image Segmentation Algorithms are based on:

Similarity Principle (Region Approach)

Objective is to group pixels based on common property to extract a coherent region

Discontinuity Principle (Boundary Approach)

Objective is to extract regions that differ in properties like intensity, color, texture etc.

Definition of Image Segmentation

An image can be portioned into many regions $R_1, R_2, R_3, \dots, R_n$

R_1	R_{21}	R_{22}
	R_{23}	R_{24}
R_3	R_4	

Example

10	10	20	20	20
10	10	20	20	20
10	10	20	20	20
30	30	20	20	20
30	30	20	20	20

10	10	20	20	20
10	10	20	20	20
10	10	20	20	20
30	30	20	20	20
30	30	20	20	20

R_1	R_3
R_2	

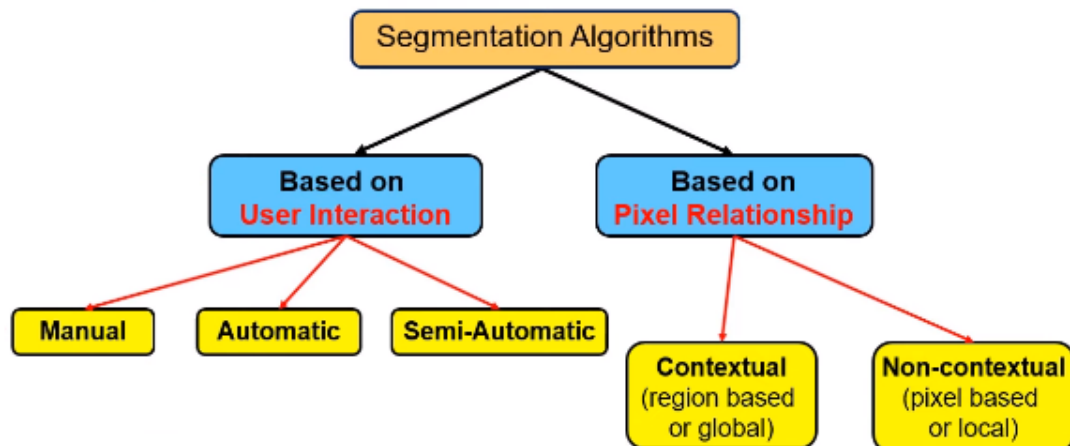
Characteristics of Segmentation Process

Let **R** represent the entire image region and **Segmentation is partitioning R** into **n** subgroups R_i

- $\bigcup_{i=1}^n R_i = R$
- R_i should be connected region $i=1,2,3,\dots,n$
- $R_i \cap R_j = \emptyset$ (for all i and j): $i \neq j$
- $P(R_i) = \text{TRUE}$ for $i = 1,2,3, \dots, n$
- $P(R_i \cup R_j) = \text{FALSE}$ for $i \neq j$

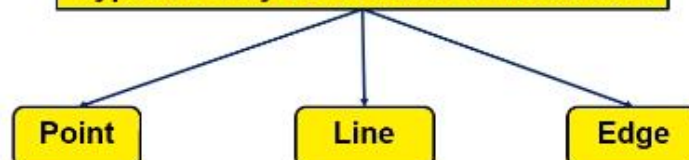
Here $P(R_i)$ is a **predicate** that indicates **some property over the region**

Classification of Image Segmentation Algorithms



Detection of Discontinuities

Types of Grey Level discontinuities are:



Point Detection

An isolated point is a point whose grey level is significantly different from its background in a homogeneous area

			3x3		
w_1	w_2	w_3	z_1	z_2	z_3
w_4	w_5	w_6	z_4	z_5	z_6
w_7	w_8	w_9	z_7	z_8	z_9

Mask

Image

Response of the mask:

$$R = \sum_{i=1}^9 w_i z_i$$

If,
 $|R| \geq T$, a point is detected
 where,
 T is a non negative integer

-1	-1	-1
-1	8	-1
-1	-1	-1

Sample Mask for Point Detection

Line Detection

In line detection, **four types of masks** are used to get the responses i.e, R_1, R_2, R_3 and R_4 for the directions vertical, horizontal, $+45^\circ$ and -45° respectively

↓																																							
<table><tr><td>-1</td><td>-1</td><td>-1</td></tr><tr><td>2</td><td>2</td><td>2</td></tr><tr><td>-1</td><td>-1</td><td>-1</td></tr></table>	-1	-1	-1	2	2	2	-1	-1	-1	<table><tr><td>-1</td><td>2</td><td>-1</td></tr><tr><td>-1</td><td>2</td><td>-1</td></tr><tr><td>-1</td><td>2</td><td>-1</td></tr></table>	-1	2	-1	-1	2	-1	-1	2	-1	<table><tr><td>-1</td><td>-1</td><td>2✓</td></tr><tr><td>-1</td><td>✓2</td><td>-1</td></tr><tr><td>✓2</td><td>-1</td><td>-1</td></tr></table>	-1	-1	2✓	-1	✓2	-1	✓2	-1	-1	<table><tr><td>②</td><td>-1</td><td>-1</td></tr><tr><td>-1</td><td>②</td><td>-1</td></tr><tr><td>-1</td><td>-1</td><td>②</td></tr></table>	②	-1	-1	-1	②	-1	-1	-1	②
-1	-1	-1																																					
2	2	2																																					
-1	-1	-1																																					
-1	2	-1																																					
-1	2	-1																																					
-1	2	-1																																					
-1	-1	2✓																																					
-1	✓2	-1																																					
✓2	-1	-1																																					
②	-1	-1																																					
-1	②	-1																																					
-1	-1	②																																					
Horizontal	Vertical	+45°	-45°																																				

Response of the mask:

$$R_k = \sum_{k=1}^4 w_k z_k$$

If, at a certain point in the image, $|R_i| > |R_j|$ for all $j \neq i$, that **point** is said to be more likely associated with a **line** in the direction of mask i

Edge Detection

- ❑ An edge is a set of connected pixels that lies on the boundary between two regions which differ in grey value. Pixels on edge is known as **edge points**
- ❑ Edges provide an outline of the object

In physical plane, **edge corresponds** to the **discontinuities in** depth, Surface orientation, change in material properties, light variations etc.

- ❖ It locates sharp changes in the intensity function
- ❖ Edges are pixels where brightness changes abruptly
- ❖ An edge can be extracted by computing the derivative of the image function
 - ✓ **Magnitude of the derivative**, indicates the strength or contrast of edge
 - ✓ **Direction of the derivative vector**, indicates the edge orientation

Edge Detection

Some of the commonly encountered edges in image processing are:

Step Edge



An abrupt change in intensity

Ramp Edge



A slow and gradual change in intensity

Spike Edge



Quick change or immediately returns to original intensity level

Roof Edge



It is not instantaneous over short distance

Stages in Edge Detection



It involves smoothing. This stage may be performed explicitly or implicitly

It distinguishes the edge pixels from other pixels

$$\text{Vector: } \nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \partial f / \partial x \\ \partial f / \partial y \end{bmatrix}$$

$$\text{Magnitude: } \nabla f = \text{mag}(\nabla f) = [G_x^2 + G_y^2]^{1/2} \approx |G_x| + |G_y|$$

$$\text{Direction of gradient: } \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

It determine the exact location of edge

Edge Detection

Edge Detection Algorithms

Derivative Types

It uses differentiation technique for edge detection

Template Matching

It uses templates that resembles the target shapes and match with image

Gaussian Derivatives

Very effective for real time images

Pattern Fit Approach

Here surface is considered as topographic surface with pixel value representing altitude

First Order Edge Detection Operators

- Local transitions among different image intensities constitute an edge
- Therefore the objective is to measure the **intensity gradient**
- Edge detectors can be viewed as **gradient calculators**

Gradient Operator is represented as:

$$\text{Vector: } \nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \partial f / \partial x \\ \partial f / \partial y \end{bmatrix}$$

$$\text{Magnitude: } \nabla f = \text{mag}(\nabla f) = \left[G_x^2 + G_y^2 \right]^{1/2} \approx |G_x| + |G_y|$$

$$\text{Direction of gradient: } \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

❖ An edge can be extracted by computing the derivative of the image function

- ✓ **Magnitude of the derivative**, indicates the strength or contrast of edge
- ✓ **Direction of the derivative vector**, indicates the edge orientation

Backward Difference:

$$= [f(x) - f(x-\Delta x)] / \Delta x$$

Forward Difference:

$$= [f(x+\Delta x) - f(x)] / \Delta x$$

Central Difference:

$$= [f(x+\Delta x) - f(x-\Delta x)] / 2\Delta x$$

These differences can be obtained by applying the following masks, assuming $\Delta x=1$:

$$\begin{aligned} \text{Backward Difference} &= f(x) - f(x-1) \\ &= [1 \quad -1] \end{aligned}$$

$$\begin{aligned} \text{Forward Difference} &= f(x+1) - f(x) \\ &= [-1 \quad +1] \end{aligned}$$

Robert Operator

- Robert Kernels are derivatives with respect to the diagonal elements
- They are known as **Cross-Gradient Operators**
- They are based on **Cross Diagonal differences**

Generic gradient based algorithm can be:

- Read the image and smooth it
- Convolve the image f with g_x
- Convolve the image f with g_y
- Compute the edge magnitude and edge orientation
- Compare the edge magnitude with a threshold value
 - If edge magnitude is higher, assign it as a possible edge point

Let $f(x, y)$ & $f(x+1, y)$ be neighbouring pixels, then
 $\frac{\partial f}{\partial x} = f(x+1, y) - f(x, y)$

Robert masks may be of:

G_x	1	0
	0	-1

G_y	0	1
	-1	0

This algorithm can be applied to other masks also

Prewitt Operator

The Prewitt Method takes the central difference of the neighbouring pixels;
 This difference can be represented mathematically as:

$\frac{\partial f}{\partial x} = [f(x+1) - f(x-1)] / 2$ $\Rightarrow$ For 2D $\rightarrow (x, y)$
 $[f(x+1, y) - f(x-1, y)] / 2$
 central difference is obtained by mask $\rightarrow [-1 \ 0 \ +1]$

G_x	-1	-1	-1
	0	0	0
	1	1	1

G_y	-1	0	1
	-1	0	1
	-1	0	1

3x3 digital approximation of Prewitt Operator is given as:

$$G_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$G_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Additional mask can be used to detect edges in diagonal as:

0	1	1
-1	0	1
-1	-1	0

-1	-1	0
-1	0	1
0	1	1

$$\nabla f \approx |G_x| + |G_y|$$

Sobel Operator

- It provides both a differentiating and a smoothing effect
- Sobel Operator relies on the central differences
- It can be viewed as an approximation of first Gaussian Derivative
- Here convolution is both commutative and associative

$$\frac{\partial}{\partial x} (f * G) = f * \frac{\partial}{\partial x} (G)$$

G_{1x}	<table><tr><td>-1</td><td>-2</td><td>-1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>2</td><td>1</td></tr></table>	-1	-2	-1	0	0	0	1	2	1	G_{1y}	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-2</td><td>0</td><td>2</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-2	0	2	-1	0	1
-1	-2	-1																			
0	0	0																			
1	2	1																			
-1	0	1																			
-2	0	2																			
-1	0	1																			

3x3 digital approximation of Sobel Operator is given as:

$$G_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$G_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

$$\nabla f \approx |G_x| + |G_y|$$

Additional mask can be used to detect edges in diagonal as:

0	1	2
-1	0	1
-2	-1	0

-2	-1	0
-1	0	1
0	1	2

Template Matching Masks

- Gradient masks are isotropic and insensitive to direction
- Sometimes it is necessary to design **direction sensitive filters**, such type of filters are known as **Template Matching Filters**

Kirsch Masks

- ✓ Kirsch masks are known as **compass masks** because they are obtained by taking one mask and rotating it to the eight major directions
- ✓ Directions are: East, North East, North, North West, West, South West, South and South East

M ₀	M ₁	M ₂	M ₃
$\begin{bmatrix} -3 & -3 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & 5 \end{bmatrix}$	$\begin{bmatrix} -3 & 5 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & -3 \end{bmatrix}$	$\begin{bmatrix} 5 & 5 & 5 \\ -3 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix}$	$\begin{bmatrix} 5 & 5 & -3 \\ 5 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix}$
M ₄	M ₅	M ₆	M ₇
$\begin{bmatrix} 5 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & -3 & -3 \end{bmatrix}$	$\begin{bmatrix} -3 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & 5 & -3 \end{bmatrix}$	$\begin{bmatrix} -3 & -3 & -3 \\ -3 & 0 & -3 \\ 5 & 5 & 5 \end{bmatrix}$	$\begin{bmatrix} -3 & -3 & -3 \\ -3 & 0 & 5 \\ -3 & 5 & 5 \end{bmatrix}$

Template Matching Masks

Robinson Compass Masks

The spatial masks for the Robinson edge operator for all the directions are as:

- ✓ Similar to Kirsch masks, the mask that produces the maximum value defines the **direction of the edge**

$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$	$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix}$
→	↗	↑	↖
$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 1 & -1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & -2 \\ -1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}$
←	↙	↓	↘

Template Matching Masks

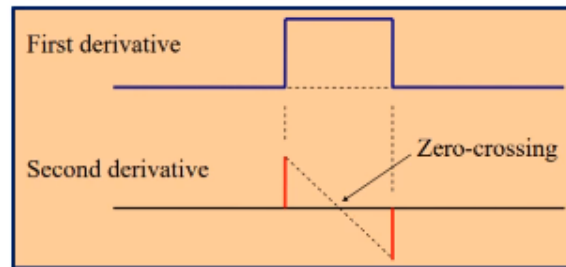
Frei-Chen Masks

- Any image can be considered as the **weighted sum** of the nine Frei-Chen masks
- Weights are obtained by a process known as **projecting process** by overlaying a 3×3 image onto each mask and by **summing the multiplication of coincident terms**
- First four masks represent the **edge space**, next four represent the **line subspace** and last one represent the **average subspace**

$$\begin{aligned}
 W_1 &= \frac{1}{2\sqrt{2}} \begin{bmatrix} 1 & \sqrt{2} & 1 \\ 0 & 0 & 0 \\ -1 & -\sqrt{2} & -1 \end{bmatrix} & W_2 &= \frac{1}{2\sqrt{2}} \begin{bmatrix} 1 & 0 & -1 \\ \sqrt{2} & 0 & -\sqrt{2} \\ 1 & 0 & -1 \end{bmatrix} \\
 W_3 &= \frac{1}{2\sqrt{2}} \begin{bmatrix} 0 & -1 & \sqrt{2} \\ 1 & 0 & -1 \\ -\sqrt{2} & 1 & 0 \end{bmatrix} & W_4 &= \frac{1}{2\sqrt{2}} \begin{bmatrix} \sqrt{2} & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & -\sqrt{2} \end{bmatrix} \\
 W_5 &= \frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \end{bmatrix} & W_6 &= \frac{1}{2} \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} & W_9 &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \\
 W_7 &= \frac{1}{6} \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix} & W_8 &= \frac{1}{6} \begin{bmatrix} -2 & 1 & -2 \\ 1 & 4 & 1 \\ -2 & 1 & -2 \end{bmatrix}
 \end{aligned}$$

Second Order Derivative Filters

- ❑ In first derivative, edges are considered to be present when edge magnitude is large compared to the threshold value
- ❑ In case of **second derivative**, edge is present at that location where the **second derivative is zero**
- ❑ It is like zero crossing, which can be observed as a sign change



Laplacian Operator

- ❑ Laplacian Algorithm is one of the **zero-crossing algorithm**
- ❑ Laplacian masks are **very sensitive to noise** because there is no magnitude checking, even a small ripple looks like edge
- ❑ Therefore image must be filter first and then edge detection process is applied
- ❑ Advantage of Laplacian operator is that they are **rotationally invariant**

$$\nabla^2 f(x, y) = \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2}$$

The Laplacian of a 2D function $f(x, y)$ is a 2nd order derivative is defined as:

$$\frac{\partial^2 f(x, y)}{\partial x^2} = f(x+1, y) - 2f(x, y) + f(x-1, y)$$

$$\frac{\partial^2 f(x, y)}{\partial y^2} = f(x, y+1) - 2f(x, y) + f(x, y-1)$$

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

Laplacian Operator

Algorithm can be written as:

- Generate the mask
- Apply the mask
- Detect the zero crossing
 - ✓ Zero crossing is a situation where pixels in neighbourhood differ from each other pixel in sign

0	1	0	1	1	1	0	0	-1	0	0
1	-4	1	1	-8	1	-1	-2	16	-2	-1
0	1	0	1	1	1	0	-1	-2	-1	0
						0	0	-1	0	0

Different Laplacian Masks

0	-1	0	-1	-1	-1
-1	4	-1	-1	8	-1
0	-1	0	-1	-1	-1

$$\nabla^2 f = 4z_5 - (z_2 + z_4 + z_6 + z_8)$$

for other mask,

$$\nabla^2 f = 8z_5 - (z_1 + z_2 + z_3 + z_4 + z_6 + z_7 + z_8 + z_9)$$

More Advanced Techniques for Edge Detection

- ❑ The basic edge detection method is based on simple filtering without taking note of image characteristics and other information
- ❑ More advanced techniques make attempt to improve the simple detection by taking into account factors such as noise, scaling etc.

Laplacian of Gaussian (Marr-Hildreth) Operator

Discussion of Marr and Hildreth was about:

- ❖ Intensity of changes is not independent of image scale
- ❖ Sudden intensity change will cause a zero crossing of the second derivative

Therefore, an edge detection operator should:

- ❖ Be capable of being tuned to any scale
- ❖ Be capable of computing the first and second derivatives

To minimize the noise susceptibility of the Laplacian Operator, Laplacian of Gaussian (LoG) Operator is often preferred

Hough Transform and Shape Detection

Hough transform is a feature extraction method for detecting simple shapes such as circles, lines etc. in an image

- ❖ Hough Transform takes the images created by edge detection operators but most of the time, edge map is disconnected
- ❖ Therefore Hough Transform is used to connect the disjointed edge points

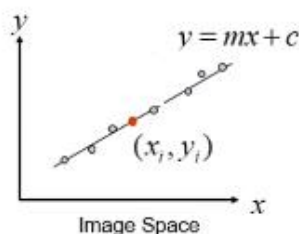


Equation of Line:

$$y = mx + c$$

Where,
 m = slope
 c = intercept of the line

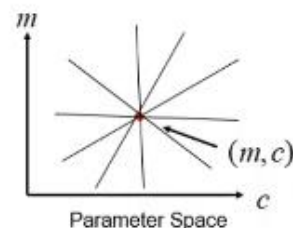
Problem is that, Infinite lines can be drawn connecting these points



Therefore an edge point in the x-y plane is transformed to the c-m

Point (x_i, y_i)

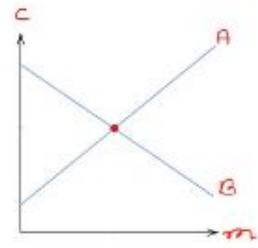
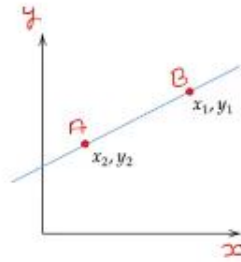
$$y_i = mx_i + c \quad \text{or} \quad c = -x_i m + y_i$$



Parameter space also called Hough Space

If A and B are two points connected by a line in spatial domain

They will be intersecting lines in Hough Space

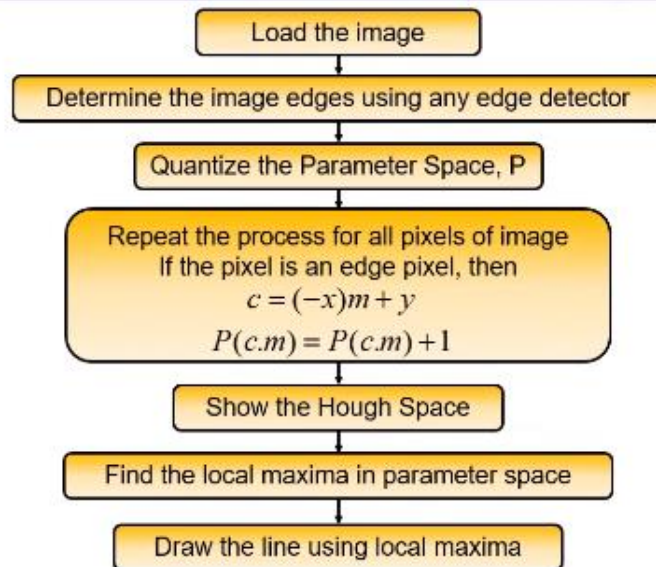


$$m^1, c^1$$

$$y = m^1 x + c^1$$

$$=$$

Algorithm



- Limitation of **previous algorithm** is that it does not work for vertical lines because they have infinity slope
- Therefore this **line** must be converted into **polar coordinates**

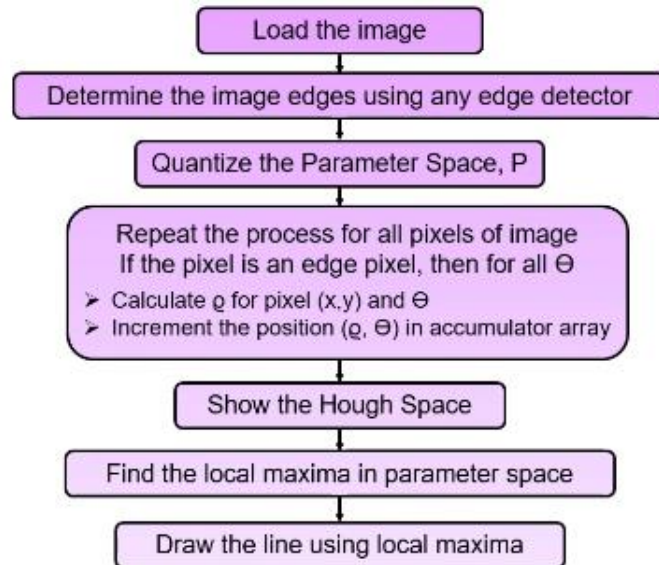
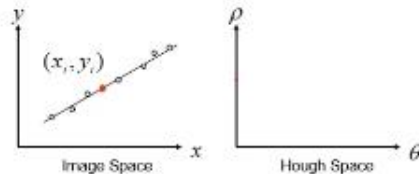
Equation of Line in polar:

$$\rho = x \cos \theta + y \sin \theta$$

Where,

θ = angle between the line

ρ = diameter



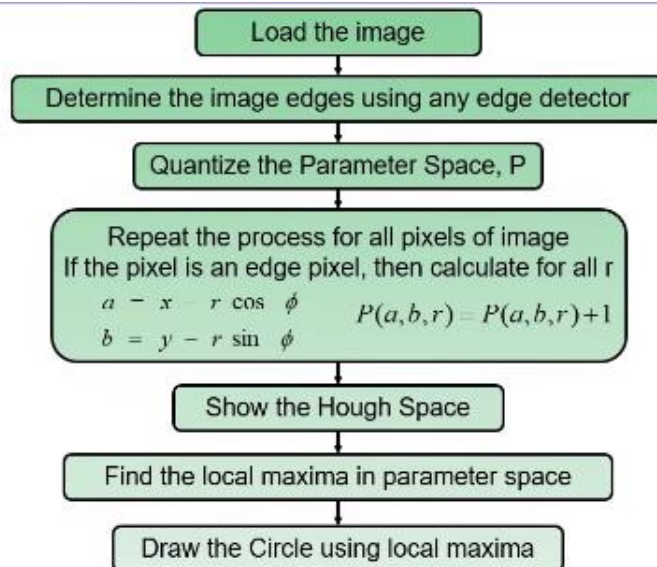
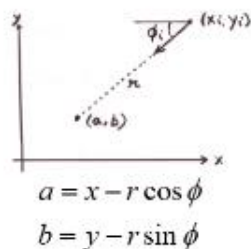
Hough Transform for other shapes can also be found

For a **Circle Detection**, it can be given as

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

Edge Location (x_i, y_i)

Edge Direction ϕ_i



Principle of Thresholding

Thresholding is an important technique for image segmentation

- ❖ It produces uniform regions based on the threshold criteria, T
- ❖ Key parameter of thresholding process is the choice of threshold value

- ❖ If thresholding operation depends upon only grey scale value, it is known as **Global Thresholding**
- ❖ In case neighborhood properties (or some local properties) is also taken into account, method is known as **Local Thresholding**
- ❖ If case T depends on pixel coordinates also, it is known as **Dynamic (or Adaptive) Thresholding**

Thresholding is a function of:

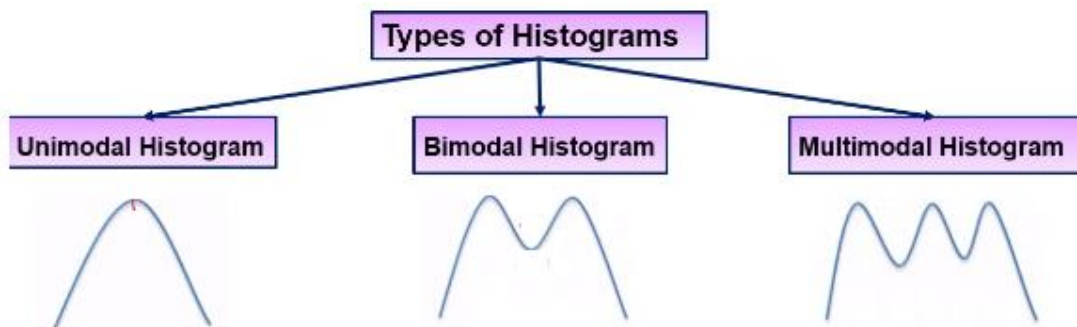
- Spatial Coordinates i.e (x,y)
- Grey level of the pixel i.e $f(x,y)$
- Some local property of the image i.e $A(x,y)$

Therefore, thresholding operation can be expressed as:

$$Th = T[x,y, f(x,y), A(x,y)]$$

Histogram and Thresholding

- The quality of thresholding algorithm depends on the selection of a suitable threshold
- Tool that helps to find the threshold is histogram



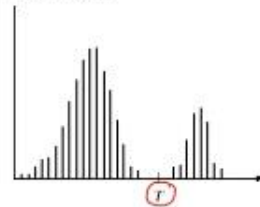
Histogram and Thresholding

Suppose that an image, $f(x,y)$, is composed of light objects on a dark background, and the shown figure is the histogram of the image

Single Level Thresholding

The objects can be extracted by comparing pixel values with a threshold T

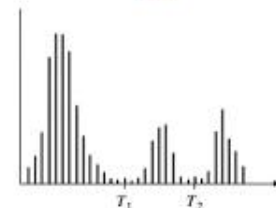
at any point (x,y) if $\begin{cases} f(x,y) > T, \text{ it is called object} \\ f(x,y) < T, \text{ it is called background} \end{cases}$



Multi Level Thresholding

It is also possible to extract objects that have a specific intensity range using multiple thresholds

if $T_1 < f(x,y) \leq T_2 \rightarrow$ one object class
 $f(x,y) > T_2 \rightarrow$ other object class



Global Thresholding

- ❑ In bimodal images, histogram have two distinct peaks separated by a valley between them
- ❑ Valley point is chosen as threshold (T)
- ❑ Then pixels of given image are compared with threshold

Threshold process is given as:

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) \geq T \\ 0 & \text{otherwise} \end{cases}$$

The basic global threshold (T) is calculated as:

- 1) Randomly select an initial threshold T
- 2) Segment the image using two group of pixels G_1 and G_2
- 3) Determine mean (m_1) of the pixels (in G_1 group) that lie below T in histogram
- 4) Determine mean (m_2) of the pixels (in G_2 group) that lie above T in histogram
- 5) New threshold is:

$$T_{\text{new}} = (m_1 + m_2)/2$$
- 6) Repeat the step no.2-5 until the difference in T in successive iterations is less than a predefined limit

Multiple Thresholding

- ❑ Multilevel thresholding is a process that segments a gray level image into several distinct regions
- ❑ It is the **extension** of **simple thresholding technique**

$f_i \rightarrow$ value of input image pixel

$t_1, t_2, t_3, \dots, t_n \rightarrow$ Multiple Threshold values

$g_1, g_2, \dots, g_n \rightarrow$ values of output image

$$\text{output Image} = \begin{cases} g_1 & \text{if } 0 \leq f_i \leq t_1 \\ g_2 & \text{if } t_1 < f_i \leq t_2 \\ \vdots & \vdots \\ g_n & \text{if } t_{n-1} < f_i < 255 \end{cases}$$

Adaptive Thresholding

- ❑ Adaptive thresholding changes the threshold dynamically over the image
- ❑ It is also known as **Dynamic Thresholding Algorithm**
- ❑ Adaptive thresholding works well in situations where image is affected by non-uniform illumination problem

Techniques to apply Thresholding algorithm

Divide image into many overlapping sub-images

Split the image into many sub regions

Some useful image statistics $\rightarrow$
 $\times \text{Mean} + c$
 $\times \text{Median} + c$
 $\times (\text{Min} + \text{Max})/2$
 $c \rightarrow \text{constant}$

Region Growing Algorithm

- ✓ ☐ Region growing is a procedure that groups pixels or sub regions into larger regions
- ✓ ☐ Logic behind region growing algorithms is the **principle of similarity**

Similarity Criteria

Principle of similarity states that a region is **coherent** if all the pixels of that region are **homogeneous**

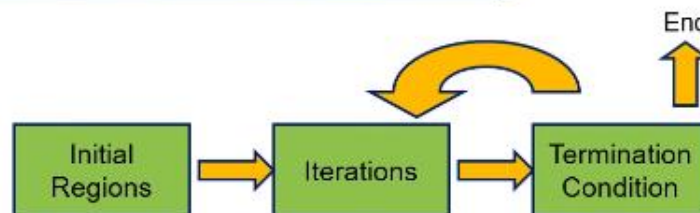
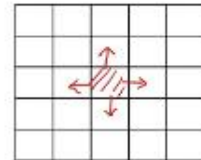
Homogeneity of regions is used as the main segmentation criterion in region growing

- gray level
- Color
- texture
- shape
- model etc.

Choice of criteria affects segmentation results dramatically!

Major steps of the region growing algorithm are:

- ☐ Selection of the initial seed
- ☐ Seed growing criteria
- ☐ Termination of the segmentation process



- Groups pixels into larger regions
- Starts with a **seed** region
- **Grows** region by **merging** neighboring pixels

- Iterative process
- How to start?
- How to iterate?
- When to stop?

Example: For a given image, show the results of region growing algorithm

* Seed point → s_1 s_2 5x5

* Threshold, T $T \leq 4$

* $s_1 = 9$

$$|f(x, y) - f(x', y')| \leq 4$$

$$|f(x, y) - 9| \leq 4$$

$$f(x, y) = \{5, 6, 7, 8, 9\} \Rightarrow \textcircled{A}$$

* $s_2 = 1$

$$|f(x, y) - f(x', y')| \leq 4$$

$$|f(x, y) - 1| \leq 4$$

$$f(x, y) = \{0, 1, 2, 3, 4, 5\} \Rightarrow \textcircled{B}$$

1	0	7	8	7
0	1	8	9	8
0	0	7	9	8
0	1	8	8	9
1	2	8	8	9

↑

Resultant Image

B	B	A	A	A
B	B	A	A	A
B	B	A	A	A
B	B	A	A	A
B	B	A	A	A

Split and Merge Algorithm

- It is an **alternative method** of image segmentation
- An image is **sub-divided** into **arbitrary disjointed region**
- Arbitrary regions can be split and merged in order to satisfy the condition

The algorithm is stated in two phases

$P(R_i) = \text{False}$

Phase-I

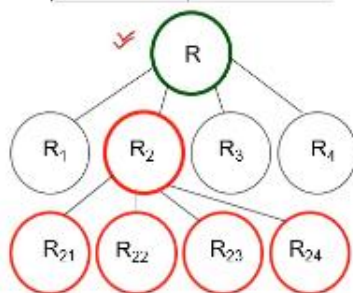
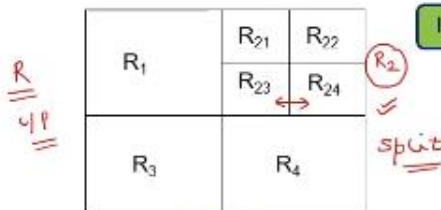
- ✓ Split & Continue subdivision process until some stopping criteria is fulfilled
- ✓ Often it is stopped when no further splitting is possible

Phase-II

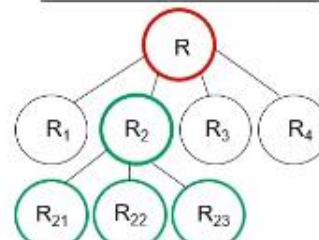
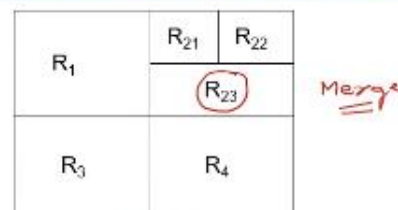
- ✓ Merge adjacent regions if the regions share any common criteria
- ✓ Stop the process when no further merging is possible

$P(R_i \cup R_j) = \text{True}$

It is also known as "Divide and Conquer" or "Top Down" approach



Quadtree of Splitting



Quadtree of Splitting and Merging

Example: For a given image, apply split and merge algorithm

Solution:

Threshold, $T \leq 3$

max. Pixel = 7
min. Pixel = 0 $\Rightarrow 7 - 0 = 7$

condition does not satisfy $\rightarrow$ split

7
4

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

3
0

splitting

Merge

R₁

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

R₂

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

max (1) - min (2)
max (2) - min (1)
 $T \leq 3$